

# Memo Field Analysis for Read-Only Field Preservation

---

**Date:** May 20, 2026

**Analysis:** QB 2021 Schema + Exported Data

**Purpose:** Determine if we can preserve “Name” and other read-only fields by mapping them to “Memo”

---

## Executive Summary

---

✅ Joseph’s preservation strategy is **VIABLE** for 4 out of 5 failing transaction types.

### Supported:

- ✅ CheckAdd — Memo field available (max 4095 chars)
- ✅ DepositAdd — Memo field available
- ✅ JournalEntryAdd — Memo field available (both transaction-level AND line-level)
- ✅ SalesReceiptAdd — Memo field available

### Not Supported:

- ❌ TransferAdd — No Memo field documented in QBXML SDK

**Current Behavior:** The code excludes “Name” from all transaction headers (via `TransactionHeader-ExcludedFields`), causing data loss. The “Name” field often contains transaction numbers, descriptions, or other identifiers that would be valuable to preserve.

---

## Part 1: QBXML Schema Analysis

---

### 1.1 CheckAdd — ✅ MEMO FIELD AVAILABLE

#### Field Specifications:

- **Field Name:** `<Memo>`
- **Data Type:** STRTYPE (string)
- **Max Length:** 4095 characters (QuickBooks Desktop), 4000 characters (QBOE)
- **Required:** Optional
- **QBXML Version:** Requires 3.0+
- **Location:** Directly under `<CheckAdd>` tag

#### Example Structure:

```
<CheckAdd>
  <Memo>Check to ABC Vendor - originally imported as Check #12345</Memo>
  <!-- other fields -->
</CheckAdd>
```

**Source:** QBXML Schema `qbxmllops20.xml`, multiple Stack Overflow examples

---

## 1.2 DepositAdd — MEMO FIELD AVAILABLE

### Field Specifications:

- **Field Name:** `<Memo>`
- **Data Type:** STRTYPE (string)
- **Max Length:** Not explicitly documented (likely same as Check - 4095 chars)
- **Required:** Optional
- **Location:** Directly under `<DepositAdd>` tag

### Example Structure:

```
<DepositAdd>
  <TxnDate>2008-09-05</TxnDate>
  <DepositToAccountRef>
    <ListID>280002-1198789618</ListID>
  </DepositToAccountRef>
  <Memo>Test Deposit</Memo>
  <DepositLineAdd>
    <PaymentTxnID>B12C3-1205600082</PaymentTxnID>
  </DepositLineAdd>
</DepositAdd>
```

**Source:** Experts Exchange QB Enterprise 5.0 documentation, QBXML examples

## 1.3 JournalEntryAdd — MEMO FIELD AVAILABLE (DUAL LOCATION)

### Field Specifications:

- **Field Name:** `<Memo>`
- **Data Type:** STRTYPE (string)
- **Max Length:** Not explicitly documented
- **Required:** Optional
- **Location:** TWO locations:
  1. **Transaction-level:** Directly under `<JournalEntryAdd>` tag
  2. **Line-level:** Within each `<JournalDebitLine>` and `<JournalCreditLine>`

### Example Structure:

```
<JournalEntryAdd>
  <TxnDate>2024-01-15</TxnDate>
  <Memo>This is a transaction-level memo</Memo>
  <JournalDebitLine>
    <AccountRef><FullName>Bank Account</FullName></AccountRef>
    <Amount>1000.00</Amount>
    <Memo>Adjustment for the bank account</Memo>
  </JournalDebitLine>
  <JournalCreditLine>
    <AccountRef><FullName>Expense Account</FullName></AccountRef>
    <Amount>1000.00</Amount>
    <Memo>This is a test</Memo>
  </JournalCreditLine>
</JournalEntryAdd>
```

**Note:** This is the MOST flexible option — we can preserve Name at transaction level AND preserve line-item descriptions.

**Source:** Intuit Developer JournalEntryAdd documentation, GitHub QBXML\_SDK\_Samples

---

## 1.4 SalesReceiptAdd — MEMO FIELD AVAILABLE

### Field Specifications:

- **Field Name:** `<Memo>`
- **Data Type:** STRTYPE (string)
- **Max Length:** Not explicitly documented
- **Required:** Optional
- **Location:** Directly under `<SalesReceiptAdd>` tag

### Example Structure:

```
<SalesReceiptAdd>
  <CustomerRef><FullName>Customer Name</FullName></CustomerRef>
  <TxnDate>2024-01-15</TxnDate>
  <Memo>Sales Receipt #12345 - Cash Sale</Memo>
  <!-- other fields -->
</SalesReceiptAdd>
```

**Source:** GitHub Gist QB SalesReceiptAdd XML, Intuit Developer documentation

---

## 1.5 TransferAdd — MEMO FIELD NOT AVAILABLE

### Field Specifications:

- **Field Name:** `<Memo>`
- **Availability:**  NOT DOCUMENTED in any QBXML SDK references
- **Structure:** TransferAdd has a simpler structure focused on:
  - `<FromAccountRef>` — source account
  - `<ToAccountRef>` — destination account
  - `<Amount>` — transfer amount (header-level, unlike other transactions)
  - `<TxnDate>` — transaction date

### Current Code Handling:

```
[ "Transfers" ] = new(StringComparer.OrdinalIgnoreCase)
{
    "Name", // not in TransferAdd schema
    "FromAccountBalance", // read-only on TransferRet
    "ToAccountBalance", // read-only on TransferRet
    // NOTE: Amount stays at header level for Transfers
},
```

**Conclusion:** For Transfers, we CANNOT preserve “Name” in a Memo field. The simplest transaction type has the least flexibility.

---

## Part 2: Exported Data Analysis

### 2.1 What's in the "Name" Field?

From `/home/ubuntu/QB-TimeWarp/Diagnostics/QB-TimeWarp-20260519.log` :

#### JournalEntries (58 total):

[WRN] Fixed empty Name **for** `JournalEntries`: generated `'ADJ - MH'`

- Many `JournalEntries` had NO Name in the export
- System generated placeholder names like "ADJ - MH"
- This suggests the original QB 2023 file also had empty Name fields

#### SalesReceipts (30 total):

[WRN] Fixed empty Name **for** `SalesReceipts`: generated `'1', '2', '3'...'30'`

- ALL 30 `SalesReceipts` had empty Name fields
- System generated sequential numbers
- This is expected — `SalesReceipts` identify by `TxnID`, not Name

#### Deposits (225 total):

[WRN] Fixed empty Name **for** `Deposits`: generated `'55', '61', '63', '65', '75', '225', '369'...`

- Many (but not all) `Deposits` had empty Name fields
- Generated names are likely `TxnNumber` values
- Some `Deposits` might have had descriptive names in QB 2023

### 2.2 Current Data Loss

#### The Problem:

1. **Export Stage:** QB 2023 returns `<Name>` in transaction Ret responses (even if empty/null)
2. **Transform Stage:** `DataTransformer.FixEmptyNameField()` generates placeholder Names for empty values
3. **Import Stage:** `TransactionHeaderExcludedFields` excludes "Name" from all transaction Add requests
4. Result:\* ANY data that was in "Name" (real or generated) is LOST

#### Impact:

- For transactions that originally HAD a Name, that descriptive text is lost
- For transactions with no Name, we lose nothing (they were empty anyway)
- But we have NO WAY to differentiate between the two cases

## Part 3: Current Code Implementation

### 3.1 ExcludedFields (Global)

From `Services/DataImporter.cs` lines ~900-915:

```
private static readonly HashSet<string> ExcludedFields = new(StringCom-
parer.OrdinalIgnoreCase)
{
    // Read-only / system-generated fields
    "ListID", "TxnID", "TimeCreated", "TimeModified", "EditSequence",
    "TxnNumber", "Balance", "TotalBalance", "Subtotal", "BalanceRemaining",
    "IsPaid", "ExternalGUID", "FullName", "OpenBalance",

    // FIX #8: TxnLineID is read-only
    "TxnLineID",

    // SDK 16.0-only fields (cause 0x80040400 in QB 2021)
    "TaxRegistrationNumber", "PreferredDeliveryMethod", "SubscriptionPaymentStatus",
    "DeliveryInfo", "TaxLineRef", "ForceUOMChange", "LinkToTxnID",

    // Payroll fields not supported in simplified QB 2021 format
    "EmployeePayrollInfo", "ClearEarnings", "BillingRateRef",
};
```

**Note:** “Name” is NOT in the global ExcludedFields — it’s handled per-transaction-type.

## 3.2 TransactionHeaderExcludedFields (Per-Transaction)

From `Services/DataImporter.cs` lines ~920-975 (FIX #8):

```

private static readonly Dictionary<string, HashSet<string>> TransactionHeaderExcluded-
Fields
    = new(StringComparer.OrdinalIgnoreCase)
{
    ["Checks"] = new(StringComparer.OrdinalIgnoreCase)
    {
        "Name",           // not in CheckAdd schema
        "Amount",         // belongs on <ExpenseLineAdd><Amount>
    },

    ["JournalEntries"] = new(StringComparer.OrdinalIgnoreCase)
    {
        "Name",           // not in JournalEntryAdd schema
        "Amount",         // belongs on each JournalDebitLine/JournalCreditLine
        "TotalAmount",    // computed rollup
        "DebitTotal",     // computed rollup
        "CreditTotal",   // computed rollup
    },

    ["Deposits"] = new(StringComparer.OrdinalIgnoreCase)
    {
        "Name",           // not in DepositAdd schema
        "Amount",         // belongs on <DepositLineAdd><Amount>
        "DepositTotal",   // computed rollup
        "TotalDeposit",   // computed rollup
    },

    ["SalesReceipts"] = new(StringComparer.OrdinalIgnoreCase)
    {
        "Name",           // not in SalesReceiptAdd schema
        "Amount",         // belongs on <SalesReceiptLineAdd><Amount>
        "TotalAmount",    // computed rollup
        "AppliedAmount",  // computed rollup
    },

    ["Transfers"] = new(StringComparer.OrdinalIgnoreCase)
    {
        "Name",           // not in TransferAdd schema
        "FromAccountBalance", // read-only on TransferRet
        "ToAccountBalance",  // read-only on TransferRet
        // NOTE: Amount stays at header level for Transfers
    },
};

```

#### Current Logic (lines ~1425-1442):

```

// — FIX #8: Per-transaction-type header-field exclusion —
if (txnHeaderExcl != null && txnHeaderExcl.Contains(prop.Name))
{
    _incompatibleFieldSkips++;
    Log.Debug("  FIX #8: Excluding '{Field}' from {EntityType} header " +
        "(not in *Add schema or belongs on a line item)",
        prop.Name, entityType);
    continue;
}

```

**Effect:** When building QBXML for these 5 transaction types, the “Name” field is skipped entirely.

## Part 4: Schema Examination — Empty Fields Arrays

### 4.1 The Missing Schema Problem

From `/home/ubuntu/QB-TimeWarp/Schemas/QB_Schema_QB_2021.json` :

```
"Transfer": {
  "EntityType": "Transfer",
  "QBXMLRequestType": "TransferQueryRq",
  "QBXMLResponseType": "TransferRet",
  "SDKVersion": "15.0",
  "Fields": [],           // ← EMPTY
  "LineItemFields": null
}
```

#### ALL transaction types have empty Fields arrays:

- Transfer: "Fields": []
- Check: "Fields": []
- Deposit: "Fields": []
- JournalEntry: "Fields": []
- SalesReceipt: "Fields": []

#### Why?

The schema extraction in `Services/SchemaExtractor.cs` likely failed to capture transaction field details. The extractor may only work for List entities (Account, Customer, Vendor, etc.), not for Transactions.

#### Impact:

We CANNOT use the schema file to determine what fields are valid for transaction \*Add requests. We must rely on:

1. QBXML SDK documentation (web search results)
2. Trial-and-error during import
3. Hard-coded knowledge in `QBXMLFieldOrdering.cs`

## Part 5: Preservation Strategy — Implementation Options

### Option 1: Field Mapping Transform (RECOMMENDED)

#### Approach:

Add a new transformation rule in `DataTransformer.cs` that maps "Name" → "Memo" BEFORE the import stage.

#### Pseudocode:

```
// In DataTransformer.cs, after FixEmptyNameField()
private static void PreserveNameInMemo(JObject entity, string entityType)
{
    // Only for transaction types with Memo support
    if (!new[] { "Checks", "Deposits", "JournalEntries", "SalesReceipts" }
        .Contains(entityType, StringComparer.OrdinalIgnoreCase))
        return;

    // If entity has a Name field
    var nameField = entity["Fields"]?["Name"];
    if (nameField == null || string.IsNullOrEmpty(nameField.ToString()))
        return;

    // Check if Memo already exists
    var memoField = entity["Fields"]?["Memo"];
    if (memoField != null && !string.IsNullOrEmpty(memoField.ToString()))
    {
        // Memo exists – prepend Name to it
        entity["Fields"]["Memo"] = $"[Name: {nameField}] {memoField}";
    }
    else
    {
        // No Memo – create one with the Name
        entity["Fields"]["Memo"] = $"[Imported Name: {nameField}]";
    }

    Log.Debug(" Preserved Name '{Name}' in Memo field for {EntityType}",
        nameField, entityType);
}
```

#### Advantages:

- Transparent to import logic — no changes needed in `DataImporter.cs`
- “Name” is still excluded from header by existing logic
- Data is preserved in a valid, writable field
- Easy to identify preserved data with `[Imported Name: ...]` prefix

#### Disadvantages:

- If existing Memo data is important, prepending Name might make it messy
- Transfers still lose Name (no Memo field)

## Option 2: Conditional Exclusion

#### Approach:

Modify `BuildAddRequestXml` to conditionally exclude “Name” only if:

1. The transaction type doesn’t support Name in \*Add schema, AND
2. The transaction type DOESN’T have a Memo field

#### Pseudocode:

```
// Modify TransactionHeaderExcludedFields setup
if (HasMemoField(entityType))
{
    // Map Name → Memo in the exclusion list
    exclusions.Remove("Name"); // Allow Name through
    // Then in BuildFieldsXml, intercept Name and rename it to Memo
}
```

**Advantages:**

- Surgical approach — only affects transactions without Memo
- Preserves existing Memo data without modification

**Disadvantages:**

- More complex logic in BuildAddRequestXml
- Requires field-name remapping during XML generation
- Still doesn't solve Transfer problem

## Option 3: Do Nothing (Current State)

**Approach:**

Accept that "Name" data is lost for transactions.

**Justification:**

- Most transactions don't have meaningful Name fields in QB 2023
- TxnID is the primary identifier
- The generated placeholder names (e.g., "1", "2", "ADJ - MH") have no value

**Advantages:**

- No code changes needed
- Existing FIX #8 already works

**Disadvantages:**

- Any real Name data that DID exist in QB 2023 is permanently lost
- No way to audit what was lost

## Part 6: Recommendations

### 6.1 Implement Option 1 (Field Mapping Transform) for 4 Transaction Types

**For: Checks, Deposits, JournalEntries, SalesReceipts**

**Why:**

- Low risk — transform happens before import, using known-good fields
- Preserves ANY data that might have been in Name (whether real or generated)
- Easy to identify preserved data with bracketed prefix
- Doesn't interfere with existing Memo content (prepends, doesn't replace)

**Implementation Steps:**

1. Add `PreserveNameInMemo()` method to `DataTransformer.cs`

2. Call it in `TransformExportedData()` after `FixEmptyNameField()` but before saving
3. Test with a single transaction of each type to verify Memo is written correctly
4. Full migration test

**Code Location:**

`Services/DataTransformer.cs` — insert around line 250 (after `FixEmptyNameField` loop)

---

## 6.2 For Transfers: Document the Loss

**For: Transfers****Why:**

- TransferAdd has NO Memo field in QBXML SDK
- No valid preservation target exists
- Transfers are the simplest transaction type — they rarely have descriptive names

**Action:**

Add a comment in `TransactionHeaderExcludedFields` explaining the limitation:

```
[ "Transfers" ] = new(StringComparer.OrdinalIgnoreCase)
{
    "Name",                // not in TransferAdd schema — NO Memo FIELD AVAILABLE
                          // so Name data cannot be preserved (TransferAdd is
                          // header-only: FromAccountRef, ToAccountRef, Amount)
    "FromAccountBalance",  // read-only on TransferRet
    "ToAccountBalance",    // read-only on TransferRet
},
```

---

## 6.3 Test with Real Data

**Test Cases:****1. Check with existing Memo:**

- Export: `<Name>Check #12345</Name><Memo>Payment for invoice 001</Memo>`
- Transform: `<Memo>[Imported Name: Check #12345] Payment for invoice 001</Memo>`
- Import: Verify QB 2021 shows both

**2. Deposit with no Memo:**

- Export: `<Name>55</Name>` (no Memo)
- Transform: `<Memo>[Imported Name: 55]</Memo>`
- Import: Verify QB 2021 shows memo

**3. JournalEntry with line-level memos:**

- Test that transaction-level Name maps to transaction-level Memo
- Verify line-level memos are NOT affected

**4. Transfer (expected loss):**

- Export: `<Name>Transfer 001</Name>`
  - Import: Verify no Memo field, Name is excluded (expected)
-

## Part 7: Summary Table

| Transaction Type        | Memo Field?  | Max Length  | Preservation Strategy           | Status      |
|-------------------------|--------------|-------------|---------------------------------|-------------|
| <b>CheckAdd</b>         | ✓ Yes        | 4095 chars  | Map Name → Memo                 | ✓ Viable    |
| <b>DepositAdd</b>       | ✓ Yes        | ~4095 chars | Map Name → Memo                 | ✓ Viable    |
| <b>JournalEntry-Add</b> | ✓ Yes (dual) | Unknown     | Map Name → Txn Memo             | ✓ Viable    |
| <b>SalesReceiptAdd</b>  | ✓ Yes        | Unknown     | Map Name → Memo                 | ✓ Viable    |
| <b>TransferAdd</b>      | ✗ No         | N/A         | <b>No preservation possible</b> | ✗ Data Loss |

## Part 8: Fields Currently Being Excluded (For Reference)

### From Global ExcludedFields:

- ListID, TxnID, TimeCreated, TimeModified, EditSequence
- TxnNumber, Balance, TotalBalance, Subtotal, BalanceRemaining
- IsPaid, ExternalGUID, FullName, OpenBalance
- TxnLineID (read-only on line items)
- SDK 16.0-only: TaxRegistrationNumber, PreferredDeliveryMethod, SubscriptionPaymentStatus, DeliveryInfo, TaxLineRef, ForceUOMChange, LinkToTxnID
- Payroll: EmployeePayrollInfo, ClearEarnings, BillingRateRef

### From TransactionHeaderExcludedFields (Per-Type):

- **All 5 types:** Name
- **Checks:** Amount (goes on ExpenseLineAdd)
- **JournalEntries:** Amount, TotalAmount, DebitTotal, CreditTotal
- **Deposits:** Amount, DepositTotal, TotalDeposit
- **SalesReceipts:** Amount, TotalAmount, AppliedAmount
- **Transfers:** FromAccountBalance, ToAccountBalance

**None of these excluded fields (except Name) contain user-entered descriptive data.** They are all:

- System-generated IDs
- Calculated totals
- Read-only balances
- SDK version-specific fields

**Conclusion:** Only “Name” is worth preserving. All other excluded fields SHOULD remain excluded.

---

## Part 9: Next Steps

---

1.  **This Analysis** — Document Memo field availability (COMPLETE)
  2.  **Implement Option 1** — Add PreserveNameInMemo transform
  3.  **Test** — Verify preserved data appears in QB 2021 Memo fields
  4.  **Document** — Update CHANGE\_LOG.md with FIX #9
  5.  **Full Migration Test** — Run against Joshs\_Gold\_Coast\_II\_2023.qbw
- 

**Analysis Complete.**

**Ready for implementation decision from Joseph.**